Docker

* Types of application

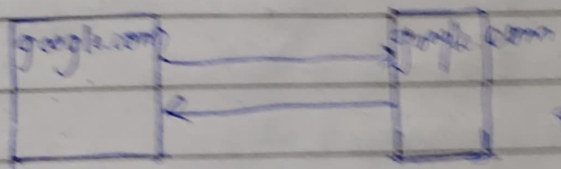
Ex:- Calculator ⇒ stand alone application (Local machine)

⇒ web - based appl^n

⇒ client - server appl^n

• Access through browser on the internet

Ex:- template, Code for Geek



Transferring information between client-server which is interactive

* Types of Application Architecture :-

⇒ Monolithic

⇒ 3- tier

⇒ Microservices.

⇒ Monolithic :- features of application are tightly coupled in single server

Ex:- Govt web application SBI etc

⇒ 3 - tier :- Ex:- Magneto, finacle

* Drawback :- * Managing the each server.
* Scaling is an issue.
* Version conflict with works on my machine.

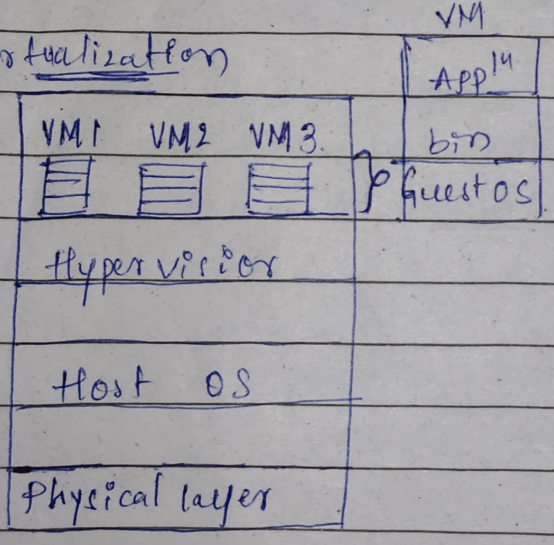
"To overcome this drawback we have Containers"

* Containers :- are portable, lightweighted, running instances of an image. It packages all the required things to run an application.
(- Source codes dependencies, plugins, executable packages --)

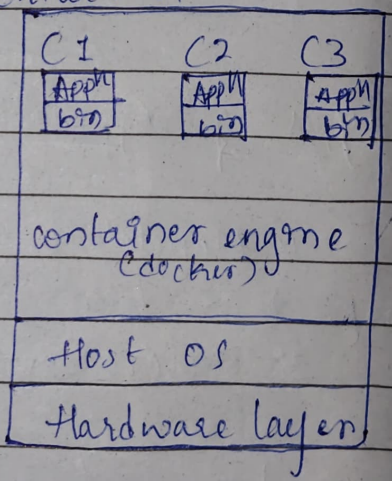
* Its property is emphesal.

"Con" are running instaces of an image.

* Virtualization



* container.

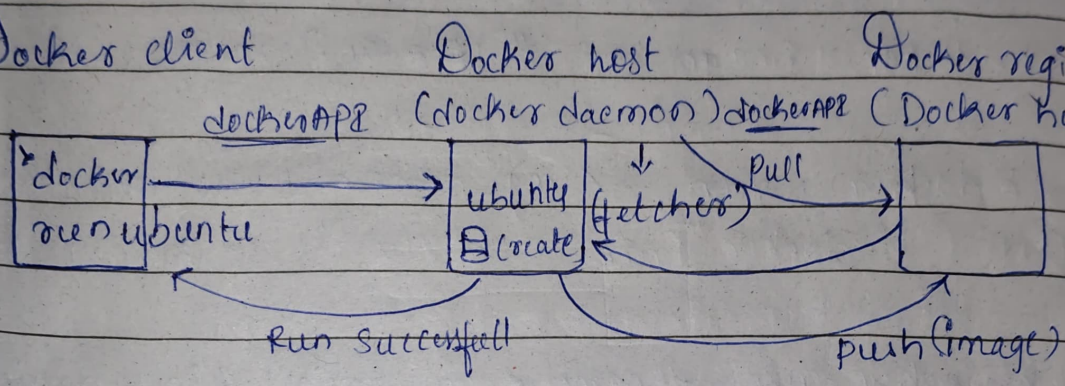


* Docker : containerization tool, used to manage, create, host /deploy/ ship the containers.

* why Docker? what are the features of Docker?

* Installing docker on windows
  - wsl V2
  > wsl -v -l # to check wsl version
  > wsl --install -d ubuntu

* Architecture of Docker



* docker files :- To customize the image.

* images :- blueprint of an appl^n's ~~package~~
  - package, source code, env.

* container :- Running instance of an image.
  - light weight, portable, running instance of an image, which isolates the application within the container (To prevent version conflict)

EC2	docker
. It provides machine layer isolation	. It provides appl^n layer isolation

Basic Docker Commands:

* Note: docker desktop is running on BG. before executing docker commands

1) > docker pull <img_name> ("ubuntu") # pull the image from hub to local machine
2) > docker images # list the images
3) > docker run <img_name> # To create docker run ubuntu container
4) > docker run --name <cont name> <img_name> # To create container with name
5) > docker run -it <img_name> # create container in interactive # it => provide pseudo terminal
6) > docker run -d <img_name> # run container in ~~detached~~ detached mode
7) > docker ps # list the running container
8) > docker ps -a # list all the containers.
9) > docker logs <container_name> (or) <cont_id> # check the logs of a container
10) > docker inspect <cont_name> # Name, subnet, which Nw, IP address it check and list the details

* How to Reduce image size

Steps:- take slim version @ compressed version

Step2: using double && in RUN command

1* * docker stop <cont_name> # stop running cont

2* * docker start <cont_name> # start stopped cont

13* * docker rm <cont_name> # delete the stopped container

14* * docker rm -f <cont_name> # forcefully delete

15* * docker rmi <img_name> # delete the image

* Docker files: It is a file where we write set of instruction to create our own image.

- Docker file instruction

→ helps to create layer in an image which runs container

1* FROM <base_img_name> # first instruction in dockerfile which provide base image/layer

ARG - to define variable

ARG default_ver: 24.04

FROM ubuntu: $default_ver

Note: Try to use less memory base image so we alpine slim.

2* WORKDIR <dir_name> # cd'ing to that dir. it will create

If also create the dir image inside that

use Absolute path

2* diff b/w copy & ADD instruction?

Step3: add .dockerignore

1) differ b/w RUN, CMD, ENTRYPOINT ?

3) COPY <source_path> <dest_path> # copies the file from local system to docker container
workdir dir
<destination>

4) RUN <command> # To execute the command while doing image creation
Ex: RUN apt update && apt install openjdk-17-jdk-y

5) CMD ["commands"] # To execute the command during container creation process.
to execute default commands which can be overwritten

6) ENTRYPOINT ["command"] # To execute the commands during container creation process.
To run executable commands which can't be overwritten

7) EXPOSE <port-no> # unblocking the port no. of container.

Note: name your dockerfile as "Dockerfile"

command to build image from dockerfile
> docker build -t <img_name> .
current dir
> docker build -f <file name> -t <img_name>
(file name)

mapping port no. ← 80

$ docker run -p <host_port> : <cont^n_port> <img_name>

publish port

* Static website hosting using dockerfile

```
FROM nginx
COPY ./ /usr/share/nginx/html
EXPOSE 80
```

$ docker stop $(docker ps -aq)

$ docker rm $(docker ps -aq)

$ docker rmi $(docker images -q)

* Short term container :

→ container executes specified task and exit / stop
→ CMD ["sh", "/bin/bash"]
→ ephemeral / temporary
Ex:- Ubuntu, java

* Long term container :

→ container runs for long time, where atleast one process will be running
Ex: nginx, tomcat
→ CMD ["nginx", "-g", "daemon off;"]

Govinda

* To Run ubuntu in long term :-

> docker run -itd ubuntu
<img-name>

* To get inside running container :-

> docker exec -it <cont.name> /bin/bash
(or) bash

* To run ubuntu base image long term

```
FROM ubuntu
CMD [ "sleep", "infinity" ].
```

* To delete all the cont images, Netw, Volume

> docker system prune.

Q write docker file for ~~Java~~ python project by taking ubuntu as base image?

=>
```
FROM python
WORKDIR /app
COPY . .
CMD [ "python", "app.py" ].
```

```
print ("Hello world")
```

Simple-python
- app.py
- Dockerfile

> docker build -t python1 .

> docker run python1

> Result :- Hello world

② python project using requirement.txt → requests

⇒

python1
- app.py
- requirements.txt
- Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

③ python flask project.

⇒

python2
- requirements.txt
- app.py
- Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirement.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"].
```

④ Django project

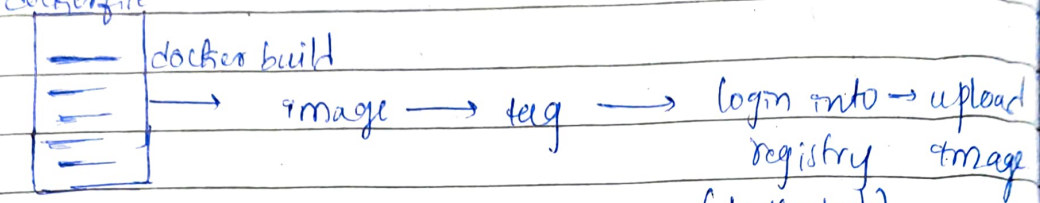
⇒

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "core/manage.py", "runserver", "0.0.0.8
000"]
```

- create containers by using image → sonarqube
- tomcat → Jenkins → maven

Docker registry

- → place to store image
- → download/pull images to LM
- → upload / push images to LM



- > docker build -t <image_name> .
- ~~> docker tag <img-name>:<version>~~
  - ↳ dockerhub username
- > docker tag <img-name> <username>/<img_name>:<version>
- > docker login -u <username>
  - (or)
  - docker login
    - username
    - Password
- > docker push <username/<img_name>:<version>

Docker Volumes.

- → persistent storage.
- independent managed by docker
- storage of local machine attaching
- local machine memory
  - temporary

Types of volumes

1) docker volume
2) Bind mounts
3) tmpfs

Commands

- docker volume ls
- docker volume create <volume>
- docker volume inspect <volume>

Note: Once we attach the volume we cannot detach if we want to detach the volume then we need to delete container. because volume is attached to file system so we cannot.

- docker run -v <volume>:/<container_path> <img_name>
- docker volume rm <vole_name>
- docker volume prune

* Bind mounts :-

- docker run -v <local_host_path>:/<contain_path> <img_name>

* Mounting container path to local machine.

Docker Networks

Is communicating with multiple devices to transfer information.

1) Bridge Network
2) Host Network
3) Overlay Network
4) None Network
5) Maclan
6) Container Namespace

Bridge → Default

container → [Docker] → host → Internet

1) Bridge network → default network (docker 0) (isolated)
2) Host network → directly connecting to local system
* container are directly mapped with host network
container → host → Internet
3) Overlay network → container can communicate multiple docker deamons
* If use swarm, kubernetes
4) None network → completly isolated no internet
No communication, No inbound & outbound
5) Mac vlan → providing Mac address to container considering the container as physical Machine
* Inside IPvlan.
6) Container Namespace: connecting one container to another container mapping
* where one container provide network to another containers
7) It is user to share Network among the container which is in same Namespace.
* If main container is deleted then other containers will be disconnected from that network

commands

1) > docker network ls

2) > docker network create -d <driver_name> <network_name>
> docker network create <network_name>

3) > docker network connect <net_name> <container>
connecting network to container
* attaching networks to existing container

* How to Reduce the size of docker image?

4) $ docker network disconnect <net_name> <cont_name>

5) $ docker network inspect <net_name>

6) $ docker run --network <net_name> <img_name>
To connect the network while creating container

7) $ docker network rm <network_name>

8) $ apt install iputils-ping -y
installing ping

Multistage Dockerfile

```
FROM ubuntu
workdir /app
COPY . .
Run apt update
Run apt install openjdk-25-jdk -y
RUN javac app.java
CMD ["java", "app.java"]
```

→ docker build -t java1 .

→ disk usage → 1.44GB

→ docker run java1

→ docker ps -as #

* This is used to Reduce the size of the image

Build stage

- SC
- dependencies
- env variable
- runtime

Production stage

- Final outcome

Build Stage → stage-1

```
FROM ubuntu AS build
WORKDIR /app
COPY test.java .
RUN apt update && apt install openjdk-21-jdk -y
    && javac test.java
```

Production stage → stage-2

```
FROM eclipse-temurin
WORKDIR /demo
COPY --from=build /app /demo
CMD ["java", "test.java"]
```

* Write multistage Dockerfile for python, flask project & To-Do project.

* tool ⇒ manage multiple container together
  - Docker compose is IAC for container
    - dependencies
    - network
    - volume
    - managing container lifecycle

  (Infrastructure as a code)

YAML
(yet another markup Language)

HTML	XML	YAML
* Hyper text Markup language	* extended markup language	* yet another markup language
* design web page	* to define artifact (versions)	* configure the things (docker compose, ansible playbook, K8S manifest file)
* pre-defined tags	* customize tags	* used key: value pair

(*) Simple docker-compose file :- (using ubuntu & Nginx)

Syntax:

```
services:
    <service-name>:
        -image: nginx
        - ports:
            - "8000:80"
        - volume:
            vol1: /<containers_path>
        -network: <network path>
```

* Commands :-

> docker-compose up # to build image & start all the services

> docker-compose up -d # to start services in detached mode

> docker-compose build # to build image

> docker-compose stop (&) start # to stop all the running container from compose file same as start

> docker-compose down
to delete containers & network from compose file

> docker-compose exec <container_name> <command>

> docker-compose ps

* apt install docker-compose plugin

> docker-compose version

* docker-compose.yml

```
services:
  ubuntu:
    image: ubuntu
    container_name: u1
    command: sleep infinity
    volumes:
      - vol1:/app
    networks:
      - busy-bee

  nginx:
    image: nginx
    container_name: n1
    ports:
      - "8000:80"
volumes:
    vol1:
networks:
    busy-bee:
      driver: bridge
```

* 2377 → cluster management
  4789 → overlay network
  7946 → Node management
  } port No.

used for docker Swarm

> Sudo apt update
> sudo apt install docker.io -y
> Sudo Systemctl status docker
> Sudo Systemctl enable docker
> Sudo Systemctl start docker
> Sudo apt install docker-compose
> docker compose --version
> docker version

> Sudo usermod -aG docker $USER
> newgrp docker

Docker Swarm

→ native clustering and container orchestration

→ Scaling
→ Self-healing
→ docker APIs
→ Clusters
→ Security

"Raft consensus database"
↳ heart beat

* Clusters:- group of machines (Nodes) is called clusters
  ↳ isolating

Node → Manager Node
→ worker nodes

* Manager Node: Schedules the tasks, gives instructions to workers defines the execution
* worker nodes: • executes the tasks/services
• container creation
→ manager can also be worker
* Services: App config
→ image, network, port mapping, volume, CPU limit containers
* task: • execution & performing

docker Swarm commands

* docker Swarm init
# initialize the current machine as manager node
→ token will be generated to create worker
* docker Swarm join-token manager
* docker Swarm join-token worker
# generate the
* docker Swarm leave
* docker Swarm leave --force
* docker node ls

> docker node inspect <node_name>

> docker node rm <node_name>

docker compose.yml

```
Version: "3.9"
services:
    nginx-Service:
        image: nginx
        ports:
            - "80:80"
        deploy:
            replicas: 3 (No of cont')
            placement:
                constraints:
                    - node.role != manager
```

where you want create cont" to manage it

> docker stack deploy -c <compose.yml> <Stack name>
                ↓ compose file

> docker services ls <Stack name>

> docker service ps <Stack name> # to Running cont"

> docker services rollback <Stack_name>

> docker service rm <Stack name>

docker Swarm

* Integrated with docker engine

→ * docker swarm overcome the problems of docker compose where it lack on managing @ orchestration and Scalability

* Native clustering and container orchestration
  → Orchestrate the container
  → Self healing (if container stops itself start new container)
  → Scaling
  → docker API (if helps to manages the clusters)

~~→ Security~~

Architecture of docker Swarm

* cluster: group of machines (nodes)
* Nodes: Machine

Manager Node → Schedules the tasks, gives instruction to workers defines the execution

worker nodes

* "Raft consensus database" → who should be manager and ~~worker~~ when to elect the manager. by nodes each manager generate heart beat.

worker node - instance & manager node

> sudo nano /etc/hostname
⇒ change to worker-node

> sudo hostnamectl set-hostname worker-node
(or) manager-node

- docker Engine on ubuntu
(apt) downloades

> sudo docker swarm init.
(or)

> sudo usermod -aG docker $USER
> newgrp docker
> docker swarm init

To add worker to [redacted] machine (or) Swarm

> docker swarm join --token <token-id> <private IP>
↓
Static IP

> docker node ls
> docker swarm join token-worker

> docker log <contain-name>

docker Swarm

* Integrated with docker engine.

→ * docker - swarm @ overcome the problem of docker - compose where it lack of managing @ orchestration and scalability

* Native clustering and container orchestration
  → Orchestrate the container
  ⇒ Self - healing (if container stops itself start new container)
  → Scaling
  → docker API (it helps to manages the clusters)

~~* Security~~

Architecture of docker swarm

• cluster: group of machines (nodes)

• Nodes: Machine

worker nodes ← Manager Node → Schedules the tasks, gives instruction to workers defines the execution

* "Raft consensus database" → who should be manager and ~~worker~~ when to elect the manager by nodes each manager generate heart beat.

* worker node : · executes the task
· container creation & orchestration
· manager can also be worker

* Services : App info
→ image, network, port mapping, container, volume, cpu limit ...

* task : · execution or performing.

docker Swarm command :- (Swarm : Group of bees)

> docker swarm init
initialize the current machine as manager node. & cluster will created
→ token will be generated to create worker

> docker swarm join-token manager
join one more manager in this cluster

> docker swarm join-token worker
generate token & add worker to

> docker swarm leave # exit from swarm

> docker swarm leave --force

> docker node ls
> docker node inspect <node_name>
> docker node rm <node-names>

} By manager

After compose file

> docker stack deploy -c <compo.yml> <stack_name>
> docker service ls <stack_name> list services
> docker services ps <stack name> # running conta
> docker service rollback <stack_name>
the container which are running will terminate & create new cont^r with old version

> docker-compose.yml

```
version: '3.9'
services:
    nginx-service:
        image: nginx
        ports:
            - "80:80"
        deploy:
            replicas: 3 #only no. of cont"
            placement:
                constraints:
                    - node.role != manager
```

to create 3 container

where you want to create cont" to manage it

> docker service ps <stack-name>

AWS

1) manager-node-instance | ubuntu | SG-80, 4946, 2377,
2) worker-node-instance | ubuntu | 4789, 22, (100-200 TCP)

=> Go to manager-node-connect
> sudo hostnamectl set-hostname manager-node
> hostname

=> Go to worker node
> sudo nano /etc/hostname (to change)
> hostname

* Install docker engine on Ubuntu copy-paste
  in both worker & manager

* Inside manager-node
  - sudo usermod -aG docker $USER
  - newgrp docker
  - docker swarm init

To add a worker to this swarm, run the following command: (token)

docker swarm join --token <SWMTKN-1-xxx xxx>
<private-IP:2377>
<port.no cluster management

- docker swarm join --token Manager.
- docker node ls & docker swarm join-token worker

* Go to worker node-instance
  - paste the token in this node instance
  - ~~sudo~~ sudo usermod -aG docker $USER
  - newgrp docker
  - docker swarm leave --force
  - docker node update --availability drain

* Manager node
  - ~~docker service create nginx-service nginx~~
    image-name service name

> docker service create --name nginx-service nginx
<service> <image>

> docker service rm nginx-service

> nano docker-compose.yml

> docker stack deploy -c docker-compose.yml webapp

> docker service ls

Drawbacks of Docker:

- Manage container
- No Self-healing property
- Auto-Scaling
- Time consuming
- No zero-down time
- Docker doesn't support monitoring tool (prometheus, grafana)
- It is ephemeral

* Orchestrating : Managing

* container Orchestrator tool:

→ Monitoring & Managing multiple container by deploying, scaling etc. in zero down time.